

5. Scikit-Learn (sklearn)

Introduction

Scikit-Learn, commonly known as **sklearn**, is one of the most widely used Machine Learning libraries in Python. It provides efficient and easy-to-use tools for building Machine Learning models for classification, regression, clustering, and dimensionality reduction. Built on top of NumPy, SciPy, and Matplotlib, it offers a consistent interface for training, testing, and evaluating models. Scikit-Learn also includes utilities for data preprocessing, feature selection, model selection, and performance evaluation. Due to its simplicity, extensive documentation, and rich collection of algorithms, Scikit-Learn is widely used in education, research, and industry for Machine Learning applications.

Import Library

```
from sklearn.model_selection import train_test_split
```

Common Functions

1. **train_test_split()** – Splits data into training and testing sets.
2. **fit()** – Trains the Machine Learning model.
3. **predict()** – Predicts output for new data.
4. **score()** – Evaluates the model performance.
5. **transform()** – Transforms input data.
6. **fit_transform()** – Fits and transforms data together.
7. **inverse_transform()** – Converts transformed data back to original.
8. **accuracy_score()** – Calculates classification accuracy.
9. **confusion_matrix()** – Generates a confusion matrix.
10. **classification_report()** – Displays classification metrics.
11. **mean_squared_error()** – Calculates Mean Squared Error.
12. **mean_absolute_error()** – Calculates Mean Absolute Error.
13. **r2_score()** – Calculates R^2 score.
14. **StandardScaler()** – Standardizes features.
15. **MinMaxScaler()** – Scales features between 0 and 1.
16. **LabelEncoder()** – Encodes categorical labels.
17. **OneHotEncoder()** – Converts categorical variables into one-hot vectors.
18. **LinearRegression()** – Creates a Linear Regression model.
19. **LogisticRegression()** – Creates a Logistic Regression model.
20. **DecisionTreeClassifier()** – Creates a Decision Tree classifier.
21. **RandomForestClassifier()** – Creates a Random Forest classifier.
22. **KNeighborsClassifier()** – Creates a KNN classifier.
23. **SVC()** – Creates a Support Vector Machine classifier.
24. **KMeans()** – Performs K-Means clustering.
25. **PCA()** – Performs Principal Component Analysis.

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    mean_squared_error,
    mean_absolute_error,
    r2_score
)

# Sample Data
X = np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
```

```

y = np.array([2,4,6,8,10,12,14,16])

# 1. train_test_split()
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
print("1. Train/Test Split Done")

# 2. fit()
model = LinearRegression()
model.fit(X_train, y_train)
print("2. Model Trained")

# 3. predict()
pred = model.predict(X_test)
print("3. Prediction:", pred)

# 4. score()
print("4. Score:", model.score(X_test, y_test))

# 5. transform()
scaler = StandardScaler()
scaler.fit(X_train)
print("5. Transform:\n", scaler.transform(X_train))

# 6. fit_transform()
scaled = scaler.fit_transform(X_train)
print("6. Fit Transform:\n", scaled)

# 7. inverse_transform()
print("7. Inverse Transform:\n", scaler.inverse_transform(scaled))

# 8. accuracy_score()
y_true = [0,1,1,0]
y_pred = [0,1,0,0]
print("8. Accuracy:", accuracy_score(y_true,y_pred))

# 9. confusion_matrix()
print("9.\n", confusion_matrix(y_true,y_pred))

# 10. classification_report()
print("10.\n", classification_report(y_true,y_pred))

# 11. mean_squared_error()
print("11. MSE:", mean_squared_error(y_test,pred))

# 12. mean_absolute_error()
print("12. MAE:", mean_absolute_error(y_test,pred))

# 13. r2_score()
print("13. R2 Score:", r2_score(y_test,pred))

# 14. StandardScaler()
std = StandardScaler()
print("14.\n", std.fit_transform(X))

# 15. MinMaxScaler()
mm = MinMaxScaler()
print("15.\n", mm.fit_transform(X))

# 16. LabelEncoder()
le = LabelEncoder()
print("16.", le.fit_transform(["Cat", "Dog", "Cat", "Bird"]))

# 17. OneHotEncoder()
ohe = OneHotEncoder(sparse_output=False)
print("17.\n", ohe.fit_transform([["Red"], ["Blue"], ["Green"]]))

# 18. LinearRegression()
lr = LinearRegression()
lr.fit(X,y)
print("18.", lr.predict([[9]]))

# 19. LogisticRegression()
X_cls = [[1],[2],[3],[4]]
y_cls = [0,0,1,1]
log = LogisticRegression()
log.fit(X_cls,y_cls)
print("19.", log.predict([[2.5]]))

# 20. DecisionTreeClassifier()
dt = DecisionTreeClassifier(random_state=0)
dt.fit(X_cls,y_cls)

```

```

print("20.", dt.predict([[3]]))

# 21. RandomForestClassifier()
rf = RandomForestClassifier(random_state=0)
rf.fit(X_cls,y_cls)
print("21.", rf.predict([[3]]))

# 22. KNeighborsClassifier()
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_cls,y_cls)
print("22.", knn.predict([[3]]))

# 23. SVC()
svc = SVC()
svc.fit(X_cls,y_cls)
print("23.", svc.predict([[3]]))

# 24. KMeans()
kmeans = KMeans(n_clusters=2, random_state=0, n_init=10)
kmeans.fit(X)
print("24. Cluster Labels:", kmeans.labels_)

# 25. PCA()
X_pca = np.array([[1,2],[2,3],[3,4],[4,5]])
pca = PCA(n_components=1)
print("25.\n", pca.fit_transform(X_pca))

```

```

[[1.]
 [8.]
 [3.]
 [5.]
 [4.]
 [7.]]
8. Accuracy: 0.75
9.
[[2 0]
 [1 1]]
10.

```

	precision	recall	f1-score	support
0	0.67	1.00	0.80	2
1	1.00	0.50	0.67	2
accuracy			0.75	4
macro avg	0.83	0.75	0.73	4
weighted avg	0.83	0.75	0.73	4

```

11. MSE: 0.0
12. MAE: 0.0
13. R2 Score: 1.0
14.
[[-1.52752523]
 [-1.09108945]
 [-0.65465367]
 [-0.21821789]
 [ 0.21821789]
 [ 0.65465367]
 [ 1.09108945]
 [ 1.52752523]]
15.
[[0.
 [0.14285714]
 [0.28571429]
 [0.42857143]
 [0.57142857]
 [0.71428571]
 [0.85714286]
 [1.
 ]]
16. [1 2 1 0]
17.
[[0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]
18. [18.]
19. [1]
20. [1]
21. [1]
22. [1]
23. [1]
24. Cluster Labels: [0 0 0 0 1 1 1 1]
25.
[[-2.12132034]
 [-0.70710678]
 [ 0.70710678]
 [ 2.12132034]]

```

Start coding or [generate](#) with AI.

